



JAVASCRIPT DOM MANIPULATION

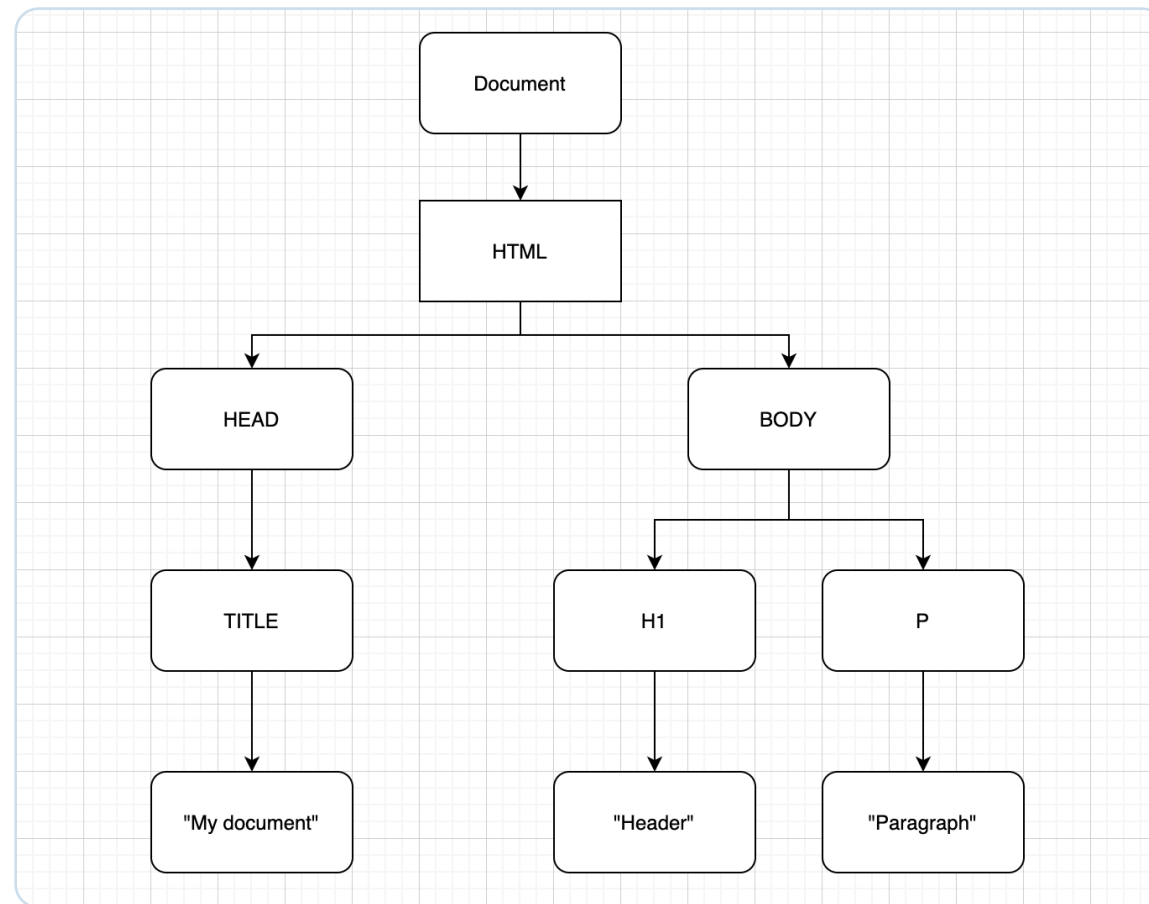
Giảng viên: Tạ Chí Hiếu

Mục tiêu bài học

- ☑ • Hiểu DOM là gì
- ☑ • Biết cách Select Elements
- ☑ • Thành thạo Creating và Modifying Elements
- ☑ • Hiểu được Manipulating Attributes và Classes
- ☑ • Hiểu Event Listeners cơ bản
- ☑

DOM là gì?

- **DOM (Document Object Model):**
Representation của HTML document như một tree of objects.
- **Structure hierarchy:**
Document → html → head/body
- **Interface:** Cầu nối giữa HTML tĩnh và JavaScript động.



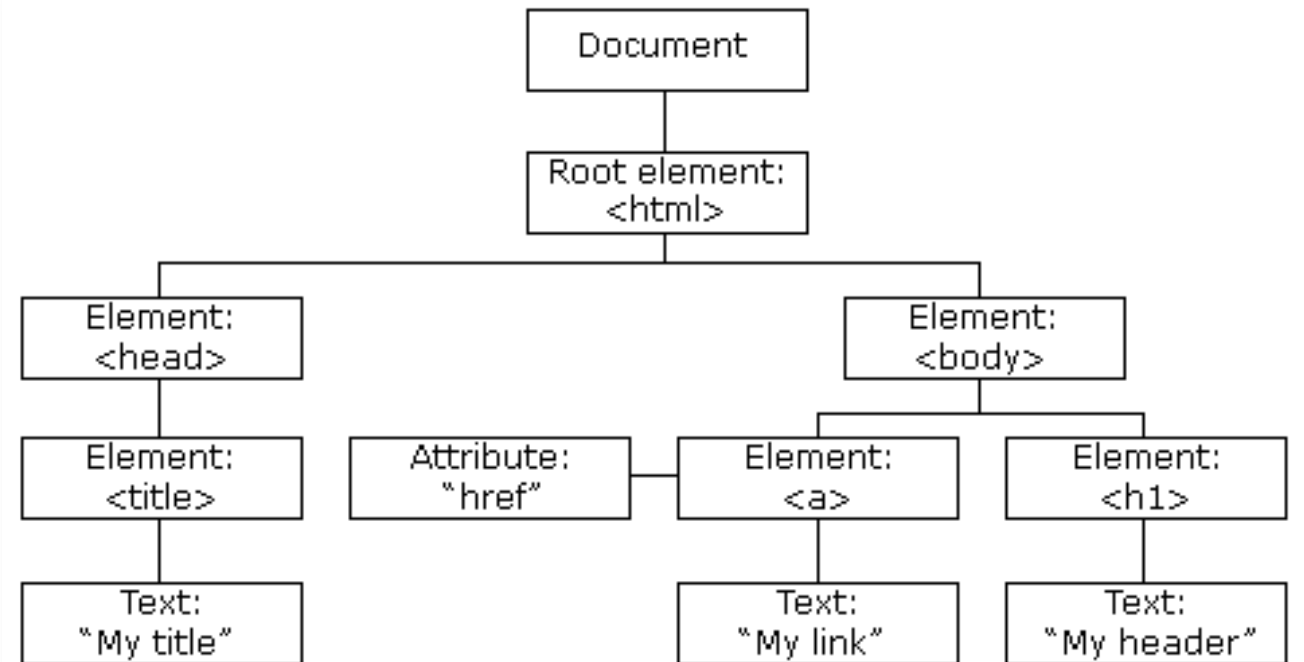
DOM Tree Visualization

Mỗi HTML element = Node trong tree

Các mối quan hệ (Relationships):

- **Parent:** Node cấp trên trực tiếp
- **Child:** Node con bên trong
- **Sibling:** Các node cùng cấp

Mapping giữa code HTML và mô hình cây (DOM tree).



Selecting Elements - `querySelector`

- **`querySelector`**: Chọn phần tử *đầu tiên* khớp với selector.
- Sử dụng cú pháp **CSS selectors** quen thuộc (`.class`, `#id`, `tag`).
- Đây là phương thức tiêu chuẩn hiện đại và linh hoạt nhất.

```
// Select first matching element
let element = document.querySelector('.my-class');
let element2 = document.querySelector('#my-id');
let element3 = document.querySelector('h1');
```

Selecting Elements - `querySelectorAll`

- **`querySelectorAll`**: Chọn *tất cả* các phần tử khớp với selector.
- Trả về một **NodeList** (đối tượng giống mảng).
- Dễ dàng lặp qua các phần tử bằng phương thức **`forEach`**.

```
// Select all matching elements
let elements = document.querySelectorAll('.my-class');
// Returns NodeList (array-like)
```

Tip: NodeList không phải là Array,
nhưng có sẵn phương thức `.forEach()`

Selecting Elements - getElementById

- **getElementById**: Chọn phần tử duy nhất thông qua ID.
- Cách tiếp cận cổ điển nhưng vẫn rất phổ biến.
- *Ưu điểm*: Hiệu năng cực kỳ nhanh.
- Trả về *element* hoặc *null* nếu không tìm thấy.

So sánh nhanh:

`getElementById('id')`

vs

`querySelector('#id')`

```
let element = document.getElementById('my-id');
```

Selecting Elements - `getElementsByClassName`

- **`getElementsByClassName`**: Chọn tất cả các phần tử có cùng class.
- Trả về một **HTMLCollection** (live collection).
- Ngày nay ít dùng hơn so với `querySelectorAll`.

```
let elements = document.getElementsByClassName('my-class');  
// Returns HTMLCollection
```

Lưu ý quan trọng: HTMLCollection là *"live"* (tự động cập nhật khi DOM thay đổi), còn NodeList từ `querySelectorAll` là *"static"*.

Modifying Elements - Text Content

textContent

- Chỉ thay đổi văn bản thuần túy.
- An toàn hơn (chống XSS).
- Tốc độ xử lý nhanh.

innerHTML

- Thay đổi nội dung HTML.
- Rủi ro bảo mật nếu dữ liệu không được kiểm soát.
- Trình duyệt phải parse lại HTML.

```
let element = document.querySelector('h1');  
element.textContent = 'New Title'; // Safe  
element.innerHTML = '<strong>Bold Title</strong>'; // Parsed
```

Creating Elements

Quy trình 3 bước:

1. **Create:** Tạo element mới trong bộ nhớ.
2. **Configure:** Thiết lập nội dung, class, id...
3. **Append:** Thêm vào cây DOM thực tế.



```
// 1. Create
let newDiv =
document.createElement('div');
// 2. Configure
newDiv.textContent = 'New element';
// 3. Append
document.body.appendChild(newDiv);
```

Removing Elements

Có hai cách chính để xóa một phần tử khỏi DOM:

```
// Modern ES6 method
let el =
document.querySelector('.item');
el.remove();
// Gọn gàng, dễ dùng
```

```
// Legacy method
el.parentNode.removeChild(el);
// Phải thông qua cha
```

Khuyến khích dùng phương thức **remove()** cho các dự án hiện đại.

Attributes - `setAttribute`, `getAttribute`

- `setAttribute(name, value)`: Thiết lập giá trị cho thuộc tính.
- `getAttribute(name)`: Lấy giá trị của thuộc tính hiện tại.
- `removeAttribute(name)`: Gỡ bỏ hoàn toàn thuộc tính.

```
let img = document.querySelector('img');  
img.setAttribute('src', 'new-image.jpg');  
img.getAttribute('src');  
img.removeAttribute('alt');
```

Attributes - Properties

- **Truy cập trực tiếp** qua dot notation (dấu chấm).
- Cú pháp: *element.property*
- *Ưu điểm*: Code ngắn gọn hơn, thực thi nhanh hơn các methods.
- Tự động map với hầu hết các thuộc tính HTML tiêu chuẩn.

```
img.src = 'new-image.jpg';  
img.alt = 'Description';
```

So sánh: `img.src` nhanh và tiện hơn `img.setAttribute('src', ...)`, nhưng `setAttribute` linh hoạt hơn với các custom attributes.

Classes - classList Methods

classList cung cấp một interface hiện đại để quản lý CSS classes:

- **add / remove:** Thêm hoặc xóa class.
- **toggle:** Đảo ngược trạng thái (có thì xóa, không có thì thêm).
- **contains:** Kiểm tra xem phần tử có class đó không.

```
let el = document.querySelector('.box');  
el.classList.add('active');  
el.classList.remove('hidden');  
el.classList.toggle('highlight');  
el.classList.contains('active'); // true/false
```

Event Listeners - addEventListener

- **addEventListener(event, callback)**: Lắng nghe tương tác của người dùng.
- Tách biệt hoàn toàn JavaScript khỏi HTML (không dùng inline *onclick*).
- Cho phép thêm nhiều listeners cho cùng một sự kiện.

```
let button = document.querySelector('#my-btn');  
// Standard function  
button.addEventListener('click', function() {  
  console.log('Button clicked!');  
});  
// Arrow function  
button.addEventListener('click', () => console.log('Clicked!'));
```

Event Listeners - Common Events

Mouse

click
mouseenter
mouseleave
dblclick



Keyboard

keydown
keyup
keypress



Form

submit
change
input
focus / blur



THỰC HÀNH - Selecting Elements

Bài tập: Thực hiện các bước sau trong file index.html của bạn:

- Tạo cấu trúc HTML đơn giản với id và class.
- Sử dụng *querySelector* để chọn một phần tử bất kỳ.
- Sử dụng *querySelectorAll* để lấy danh sách các phần tử.
- Sử dụng *getElementById* để truy cập trực tiếp qua ID.
- Kiểm tra kết quả trong tab **Console** của DevTools.



THỰC HÀNH - Modifying Elements

Bài tập: Thao tác nội dung và cấu trúc DOM:

- Thay đổi văn bản tiêu đề bằng *textContent*.
- Chèn thêm mã HTML vào một div bằng *innerHTML*.
- Sử dụng *createElement* để tạo một thẻ `<li>` mới.
- *Append* phần tử vừa tạo vào danh sách `<ul>`.
- Thử xóa một phần tử bằng phương thức *remove()*.

THỰC HÀNH - Attributes & Classes

Bài tập: Quản lý Attributes và CSS Classes:

- Thay đổi thuộc tính *src* của một thẻ `<img>`.
- Sử dụng dot notation để gán giá trị cho các properties.
- Thêm và xóa class cho phần tử bằng **`classList.add / remove`**.
- Sử dụng **`classList.toggle`** để bật/tắt một trạng thái.
- Kiểm tra sự tồn tại của class bằng *`classList.contains`*.

THỰC HÀNH - Event Listeners

Bài tập: Phản hồi tương tác người dùng:

- Gán sự kiện **click** cho một nút bấm.
- Thử nghiệm gán nhiều listeners khác nhau cho cùng 1 phần tử.
- Lắng nghe sự kiện *mouseenter* / *mouseleave*.
- Thay đổi *textContent* của một thẻ khi người dùng click vào.
- Toggle class để đổi màu nền khi click nút.

TỔNG KẾT

1. **DOM:** Mô hình cây biểu diễn tài liệu HTML của bạn.
2. **Selecting:** Dùng *querySelector* hoặc *getElementById*.
3. **Modifying:** Sử dụng *textContent* (an toàn) và *createElement*.
4. **Attributes:** Thao tác qua methods hoặc trực tiếp bằng properties.
5. **Classes:** Quản lý chuyên nghiệp bằng *classList*.
6. **Events:** Kết nối người dùng qua *addEventListener*.
7. **Best Practice:** Ưu tiên **textContent** để đảm bảo bảo mật.

HỎI ĐÁP

Câu hỏi thường gặp:

- Nên dùng `querySelector` hay `getElementById`?
- Khi nào dùng `textContent` thay vì `innerHTML`?
- Tại sao Element không hiển thị sau khi Create?
- Gán nhiều Event listeners cho 1 nút được không?
- `classList` và `className` khác nhau gì?



Next

BÀI HỌC TIẾP THEO

Bài 50: Events & Async Programming

Khám phá Event Bubbling, Delegation,
Promises và Async/Await trong JavaScript.